



CINEMATCH

An AI-Powered Movie & Game Recommender

Describe your mood — get the perfect pick, and the reason why.

Project Report

Software Engineering — Capstone Project
Sejong University

Team Members

Asliddin Ismail · Chae Hunwoo

June 2026

Table of Contents

1. Executive Summary.....	3
2. Problem and Motivation.....	3
3. Project Goals and Scope.....	3
4. System Architecture.....	4
5. How AI / the LLM Is Used.....	5
6. The C Recommendation Engine.....	5
7. Application Features.....	6
8. Technology Stack and Data Sources.....	6
9. Reliability and Resilience.....	7
10. Engineering Process.....	7
11. Challenges and Solutions.....	7
12. Results.....	8
13. Future Work.....	8
14. Conclusion.....	8
Appendix A — How to Run.....	9

1. Executive Summary

CINEMATCH is a web application that solves a small but universal problem: the time people waste deciding what to watch or play. Instead of endless scrolling, the user describes their mood in plain English — for example, “something scary but short, watching with friends” — and CINEMATCH instantly responds with one confident recommendation and a personalized explanation of why it fits.

The project is a deliberate combination of two technologies. A Large Language Model (LLM) handles natural language: it understands the user’s free-text mood and later writes a warm, human explanation of the chosen title. A program written in C performs the deterministic scoring: it evaluates a library of 55 carefully curated titles against the user’s preferences and selects the best match. This division — AI for language, C for logic — is the heart of the system and the central story of the capstone.

Beyond the core recommendation flow, the finished application grew into a four-page product: an AI Match page (the core feature), a Movies browser and a Games browser backed by large open datasets, and a Discussion page that generates a rich, AI-written analysis of any film. The entire system is organized using Domain-Driven Design and is fully version-controlled on GitHub.

2. Problem and Motivation

Streaming services and game libraries offer more choice than ever, but choice itself has become the problem. Studies of “decision fatigue” show that people often spend longer choosing what to watch than they would have spent enjoying a shorter title. Recommendation systems on existing platforms tend to optimize for engagement and watch-time rather than for a single, decisive answer to the question “what should I do tonight?”

CINEMATCH takes a different stance. It is opinionated by design: it gives one answer, not twenty rows of thumbnails, and it explains that answer in plain language so the user can trust it. The target user is anyone deciding what to watch or play this evening; for the purposes of this course, it also serves as a clear, end-to-end demonstration of a complete AI product.

3. Project Goals and Scope

3.1 Goals

- Deliver one clean, reliable recommendation flow from free-text input to an explained result.
- Use an LLM meaningfully at both ends of the pipeline (understanding and explanation).
- Keep deterministic scoring in a C engine, demonstrating systems programming alongside AI.
- Structure the codebase clearly using Domain-Driven Design so the architecture is easy to grade and extend.
- Produce a working live demo, a written report, presentation slides, and clean source code.

3.2 In Scope

- The mood-to-recommendation pipeline (AI → C engine → AI explanation).
- Browsing real movies and games from open datasets.
- An AI-generated discussion/analysis view for any movie.

3.3 Out of Scope (to avoid scope creep within a one-week deadline)

- User accounts, login, and persistent databases.
- Public internet deployment, payments, and mobile applications.

4. System Architecture

CINEMATCH follows Domain-Driven Design (DDD), an architectural style that separates the core business idea from the surrounding “plumbing.” This keeps each part of the system focused on a single responsibility and makes the codebase navigable for both human teammates and AI agents.

4.1 The DDD Layers

Layer	Responsibility	Technology
Interfaces	What users reach: the web pages and the HTTP API	Vite + React, Express
Application	Use cases: understand mood, pick a title, explain, discuss	Node.js
Domain	Core concepts and rules: Preferences, Title, the scoring	Node.js + C engine
Infrastructure	The outside world: AI client and C-engine runner	OpenRouter, child process

4.2 The Core Pipeline

The central AI Match feature follows a five-step flow:

1. The user types how they feel in plain English.
2. The LLM converts that text into five structured preferences: medium, genre, time available, mood, and social setting.
3. The Node backend runs the compiled C engine as a child process, passing the five preferences as command-line arguments.
4. The C engine scores all 55 titles and returns the single best match as JSON.
5. The LLM writes a short, personalized explanation, and the result is displayed in the browser.

Diagram of the data flow:

5. How AI / the LLM Is Used

The professor's requirement that the project use an LLM is satisfied at three distinct points in the application, each a genuine use of language understanding or generation rather than a token gesture:

- **Mood understanding:** the LLM reads the user's free text and emits structured preferences as strict JSON, turning vague human language into machine-usable data.
- **Explanation generation:** after the C engine picks a title, the LLM writes a warm, two-to-three sentence reason tailored to what the user originally said.
- **Movie discussion:** on the Discussion page, the LLM produces a full analysis of any film — its main theme, epic moments, what makes it special, the director's vision, and discussion questions.

The AI provider is OpenRouter, an OpenAI-compatible gateway, used exclusively with free models (Qwen3 and other open Chinese models). This choice keeps the project at zero cost for students while still providing strong language quality.

6. The C Recommendation Engine

The deterministic heart of CINEMATCH is a program written in C. It is deliberately rule-based and transparent: given five preferences, it assigns each title in the library a score and returns the highest. The scoring weights were tuned so that the most important factors (genre and medium) dominate, while finer factors (timing, mood, social setting, and rating) break ties.

6.1 Scoring Weights

Factor	Points	Meaning
Genre match	6	Strongest signal of taste
Medium match	5	Movie vs. Game preference
Mood match	4	Intense, light, deep, relaxed
Time exact match	3	Short / medium / long fits exactly
Time adjacent	1	One step away on the time scale
Social match	2	Solo vs. group
Rating	+rating/10	Quality tiebreaker

6.2 Engine Mode

The same C program runs in two modes. Run with no arguments, it presents an interactive five-question quiz in the terminal. Run with five arguments, it enters “engine mode”: it scores silently and prints a single line of JSON, which is exactly what the Node backend consumes. This dual design let the team test the engine by hand while also embedding it cleanly inside the web application.

```
cinematch Movie horror short intense group
```

returns, for example, the title “Get Out” with its full metadata as JSON.

7. Application Features

The finished web application is organized into four pages, reachable from a top navigation bar:

7.1 AI Match

The core feature. The user types a mood and receives one explained recommendation drawn from the curated 55-title library, powered by the full AI-plus-C pipeline described above.

7.2 Movies

A browser for thousands of real films, powered by the open TMDb (The Movie Database) dataset. Users can search by title, filter by genre, page through results, and click any film to see its poster, rating, and overview in a detail view.

7.3 Games

A parallel browser for video games, powered by the open RAWG dataset of over 800,000 titles. It offers the same search, genre filtering, pagination, and detail views, with cover art and ratings.

7.4 Discussion

A deeper analytical view. The user searches for any movie; CINEMATCH pulls its full cast (with photos), director, and writer from TMDb, then asks the LLM to generate a structured discussion: the film’s main theme and goal, its epic and memorable moments, what makes it special, the director’s vision, and a set of discussion questions.

8. Technology Stack and Data Sources

Component	Technology
Recommendation engine	C (compiled to cinematch.exe)
Backend	Node.js + Express
Frontend	Vite + React
Package manager	pnpm
AI provider	OpenRouter (free Qwen3 / open models)

Movie data	TMDB — The Movie Database (open API)
Game data	RAWG — Video game database (open API)
Curated library	55 titles in engine/titles.h
Version control	Git + GitHub

9. Reliability and Resilience

Free AI models are powerful but unreliable: they rate-limit frequently and occasionally fail. Because a live demo must never crash, CINEMATCH was engineered to degrade gracefully at every layer:

- **Multi-model fallback chain:** if the preferred model is unavailable, the system automatically tries the next free model (Qwen3 → Llama-3.3-70B → GPT-OSS-120B).
- **Automatic retry:** on a rate-limit or server error, the request is retried after a short delay.
- **Keyword fallback:** if every AI attempt fails, a local keyword matcher still derives sensible preferences, so a recommendation always appears.
- **Content fallback:** the Discussion page validates the AI response and substitutes a well-written default analysis if any field comes back empty.

10. Engineering Process

The project was built as a disciplined software-engineering exercise, not just a coding sprint. Several practices kept the work aligned and reviewable:

- **Requirements grilling:** before any code, the vision was sharpened through structured questioning, producing an agreed VISION.md.
- **Decision log:** every significant choice was recorded in an append-only DECISIONS.md (13 decisions, D-000 to D-012), so nothing was re-argued and no context was lost.
- **Ubiquitous language:** a shared GLOSSARY.md ensures the code, docs, and team use the same word for the same concept.
- **Frequent commits:** work was committed and pushed to GitHub after each phase, keeping progress safe and the history clear.

11. Challenges and Solutions

Challenge	Solution
Free AI models rate-limited mid-demo	Fallback chain + retry + keyword fallback
A chosen model was discontinued (404)	Queried OpenRouter for live free models, switched to Qwen3
Connecting C to JavaScript safely	C “engine mode” prints JSON; Node runs it as a child process

Small built-in library	Added Movies/Games browsers on open TMDb and RAWG datasets
Discussion fields sometimes empty	Validate AI output, substitute rich default content

12. Results

CINEMATCH meets every goal set out at the start. The core pipeline works end-to-end and was rehearsed successfully across multiple mood queries — for example, “relaxing cozy solo night” returns Balatro, “scary horror with a group” returns Alien, and “epic action game for hours” returns Super Mario Odyssey. The four-page application runs reliably, the architecture is clean and well-documented, and the complete source is committed to GitHub.

13. Future Work

- Personalization through optional user accounts and saved history.
- Unit tests for the domain layer using test-driven development.
- Public deployment so the app can be used outside the demo laptop.
- Expanding the AI pipeline to recommend directly from the large open datasets, not just the curated library.

14. Conclusion

CINEMATCH demonstrates that a focused idea, executed cleanly, can combine classical systems programming and modern AI into a single coherent product. By keeping deterministic logic in C and language work in an LLM, the project tells a clear engineering story while remaining genuinely useful. The result is a polished, reliable, and well-architected capstone that fulfills the course requirements and stands on its own as a product people would actually enjoy using.

Appendix A — How to Run

Prerequisites: Node.js, pnpm, and (to recompile the engine) a C compiler such as GCC. Two free API keys are needed for the Movies and Games pages (TMDB and RAWG), stored in `web/.env.local`; the AI key lives in `server/.env`.

1. Start the backend: `cd server` then `node server.js` (runs on port 3000).
2. Start the frontend: `cd web` then `pnpm dev` (runs on port 5173).
3. Open `http://localhost:5173` in a browser.

Source code: [GitHub repository](#)